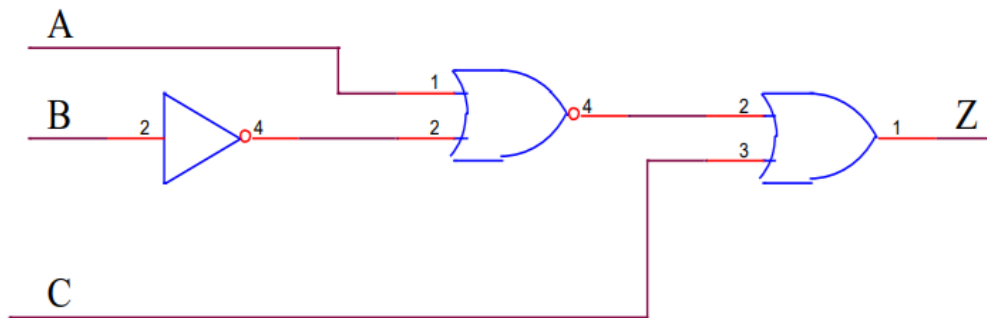


Dùng AI tạo sinh Python Code mô tả chức năng của mạch logic tổ hợp

Mạch logic tổ hợp ví dụ



Dùng AI tạo True Table:

Bảng Chân Trĩ (Truth Table)

A	B	C	$\overline{B}$	$A + \overline{B}$	$X = \overline{A + B}$ (NOR)	$Z = X + C$ (OR)
0	0	0	1	1	0	0
0	0	1	1	1	0	1
0	1	0	0	0	1	1
0	1	1	0	0	1	1
1	0	0	1	1	0	0
1	0	1	1	1	0	1
1	1	0	0	1	0	0
1	1	1	0	1	0	1

 Xuất sang Trang tính

Dùng AI tạo sinh Python code mô tả mạch logic tổ hợp

Kết quả

Mạch logic trong hình có ba đầu vào là **A**, **B**, **C** và một đầu ra là **Z**.

Phân tích Mạch Logic

1. **Đầu vào B** đi qua một cổng **NOT** (hình tam giác với vòng tròn nhỏ), cho ra $\overline{B}$.
2. **Cổng logic thứ nhất** là cổng **NOR** (cổng OR với vòng tròn nhỏ ở đầu ra). Các đầu vào của nó là **A** và $\overline{B}$.
 - Đầu ra của cổng NOR này, gọi là **X**, sẽ là $X = \overline{A + \overline{B}}$.
3. **Cổng logic thứ hai** là cổng **OR**. Các đầu vào của nó là **X** và **C**.
 - Đầu ra cuối cùng **Z** là $Z = X + C$.

Phương trình Logic

Kết hợp các bước trên, ta có phương trình logic cho đầu ra **Z**:

$$Z = \overline{A + \overline{B}} + C$$

Code Python Mô tả Mạch

Ta có thể sử dụng các phép toán logic bitwise trong Python:

- **Phép NOT** (phủ định) được mô tả bằng phép toán (hoặc nếu dùng 0 và 1).
- **Phép OR** được mô tả bằng phép toán .

Dưới đây là một hàm Python nhận ba đầu vào và trả về đầu ra :

Python

```
def logic_circuit_Z(A, B, C):
```

```
    """
```

Mô tả mạch logic tổ hợp: $Z = \text{NOT}(A \text{ OR } (\text{NOT } B)) \text{ OR } C$

A, B, C phải là giá trị boolean (True/False) hoặc nhị phân (1/0).

Trong mạch:

1. Đầu vào B đi qua cổng NOT -> not_B

2. A và not_B đi qua cổng NOR (NOT(OR)) -> X

3. X và C đi qua cổng OR -> Z

Phương trình logic: $Z = \text{not}(A \text{ or } \text{not}(B)) \text{ or } C$

"""

1. Đầu ra của cổng NOT (đầu vào B): not_B

not_B = not B

2. Đầu ra của cổng NOR (A và not_B): X = NOT(A OR not_B)

Cổng NOR là cổng OR sau đó là cổng NOT

X = not (A or not_B)

3. Đầu ra của cổng OR (X và C): Z = X OR C

Z = X or C

return Z

Ví dụ về cách kiểm tra (Sử dụng 1 cho True, 0 cho False)

print("A=0, B=0, C=0 -> Z =", logic_circuit_Z(False, False, False)) # Kết quả phải là 1 (False)

print("A=1, B=0, C=0 -> Z =", logic_circuit_Z(True, False, False)) # Kết quả phải là 0 (False)

print("A=0, B=1, C=0 -> Z =", logic_circuit_Z(False, True, False)) # Kết quả phải là 0 (True)

print("A=0, B=0, C=1 -> Z =", logic_circuit_Z(False, False, True)) # Kết quả phải là 1 (True)

print("A=1, B=1, C=0 -> Z =", logic_circuit_Z(True, True, False)) # Kết quả phải là 0 (False)

print("A=1, B=1, C=1 -> Z =", logic_circuit_Z(True, True, True)) # Kết quả phải là 1 (True)

Bạn có thể thay thế True/False bằng 1/0 nếu cần làm việc với số nguyên, nhưng việc sử dụng boolean là cách **chuẩn và rõ ràng** nhất để mô tả logic trong Python.

Kết quả chạy chương trình:

A=0, B=0, C=0 -> Z = False

A=1, B=0, C=0 -> Z = False

A=0, B=1, C=0 -> Z = True

A=0, B=0, C=1 -> Z = True

A=1, B=1, C=0 -> Z = False

A=1, B=1, C=1 -> Z = True